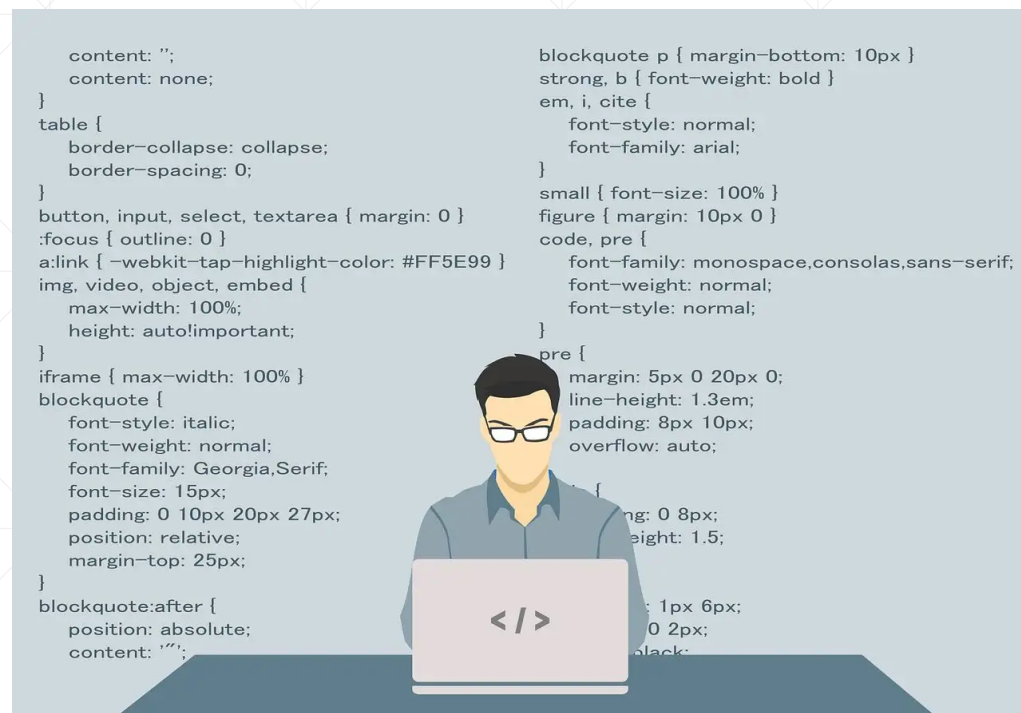


对象序列化之二



对象序列化实用编程技巧

北京理工大学计算机学院
金旭亮

序列化数组



可以直接序列化一个包容了多个对象的对象数组。其本质上就是序列化“单个对象”。因为在Java中，数组其实就是一个对象。

1 usage

```
private static void writeArray() throws IOException {  
    Person[] people = new Person[]{  
        new Person( name: "张三", age: 20),  
        new Person( name: "李四", age: 30),  
        new Person( name: "王五", age: 40)  
    };  
    try (ObjectOutputStream out = new ObjectOutputStream(  
        new FileOutputStream(dataFileName)  
    )) {  
        out.writeObject(people);  
    }  
    System.out.println("数组已经写到" + dataFileName + "中");  
}
```

示例：ReadWriteArray.java

1 usage

```
private static void readArray() throws IOException {  
    System.out.println("从" + dataFileName + "中读...");  
    try (ObjectInputStream in = new ObjectInputStream(  
        new FileInputStream(dataFileName)  
    )) {  
        Person[] people = (Person[]) in.readObject();  
        for (Person person : people) {  
            System.out.println(person);  
        }  
    } catch (ClassNotFoundException e) {  
        throw new RuntimeException(e);  
    }  
}
```

序列化多个对象实例



我们也可以调用多次`WriteObject`方法序列化多个对象，而且这些对象的类型还可以不一样，注意只需按照同样的顺序读取就行了。

```
Run: SerializeMultiObject x
"C:\Program Files\Java\jdk-17\bin\java.exe"
三个对象已经写入到文件中。
现在从文件中重建对象并输出对象的字段值
MyClass{intValue=100}
MyClass{intValue=200}
MyOtherClass{stringValue='Hello'}

Process finished with exit code 0
```

本示例将两个MyClass对象和一个MyOtherClass对象写到文件中，然后再读出来。

示例: SerializeMultiObject.java

多对象序列化的代码

写

```
1 usage
private static void writeMultiObjToFile() throws IOException {
    try( var out = new ObjectOutputStream(
        new FileOutputStream(objDataFileName))){
        var myClassObj1 = new MyClass( value: 100);
        var myClassObj2 = new MyClass( value: 200);
        var myOtherClassObj = new MyOtherClass( value: "Hello");
        out.writeObject(myClassObj1);
        out.writeObject(myClassObj2);
        out.writeObject(myOtherClassObj);
    }
    System.out.println("三个对象已经写入到文件中。");
}
```

读

```
1 usage
private static void readMultiObjFromFile()
    throws IOException, ClassNotFoundException {
    System.out.println("现在从文件中重建对象并输出对象的字段值");
    try(ObjectInputStream in = new ObjectInputStream(
        new FileInputStream(objDataFileName))){
        var myClassObj1 = (MyClass) in.readObject();
        var myClassObj2 = (MyClass) in.readObject();
        var myOtherClassObj = (MyOtherClass) in.readObject();
        System.out.println(myClassObj1);
        System.out.println(myClassObj2);
        System.out.println(myOtherClassObj);
    }
}
```



读与写的顺序需要“严格匹配”。

同一对象，序列化多次~~~

```
1 usage
private static void writePersonMultiTimes() {
    //将被多次序列化的Person对象
    Person person = new Person( name: "张三", age: 30);
    try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream(objDataFileName))) {
        for (int i = 1; i <= SerializeTimes; i++) {
            //如果是序列化多个Person对象，会得到不同的结果
            // Person person = new Person("张三", 30);
            //将同一个人对象写入输出流
            oos.writeObject(person);
            System.out.println(i + " 对象: " + person + "已写入到文件"
                + objDataFileName + "中! ");
        }
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

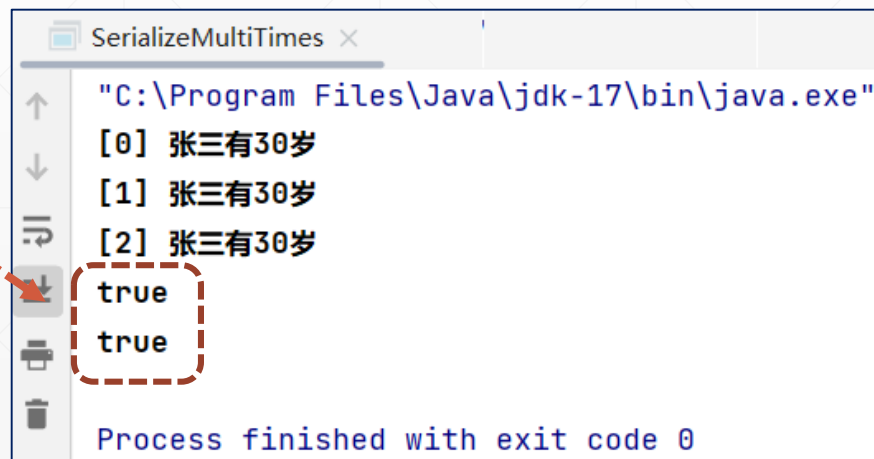
注意区分“同一个对象被序列化多次”和“相同内容的多个对象，各自被序列化一次”。

“同一对象，序列化多次”进行反序列化测试

```
private static void readPerson() {
    try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream(objDataFileName))) {
        //用一个数组来保存反序列化结果
        Person[] people = new Person[SerializeTimes];
        for (int i = 0; i < people.length; i++) {
            //从输入流中读取一个Java对象，并将其强制类型转换为Person类
            people[i] = (Person) ois.readObject();
            System.out.println "[" + i + "]" + people[i];
        }
        //检查数组中的各个元素，是否引用同一个Person对象
        System.out.println(people[0] == people[1]);
        System.out.println(people[1] == people[2]);

    } catch (IOException | ClassNotFoundException e) {
        throw new RuntimeException(e);
    }
}
```

从输出结果来看，同一个对象被序列化多次，反序列化时，并不会出现多个“副本”，Java会**只反序列化一次**，然后，**重用**这个已经反序列化好的对象。



```
SerializeMultiTimes x
"C:\Program Files\Java\jdk-17\bin\java.exe"
[0] 张三有30岁
[1] 张三有30岁
[2] 张三有30岁
true
true
Process finished with exit code 0
```


小改动，大不同！

```
private static void writePersonMultiTimes() {  
    //将被多次序列化的Person对象  
    //Person person = new Person("张三", 30);  
    try (ObjectOutputStream oos = new ObjectOutputStream(  
        new FileOutputStream(objDataFileName))) {  
        for (int i = 1; i <= SerializeTimes; i++) {  
            //如果是序列化多个Person对象，会得到不同的结果  
            Person person = new Person(name: "张三", age: 30);  
            //将person对象写入输出流  
            oos.writeObject(person);  
            System.out.println(i + " 对象: " + person + "已写入到文件"  
                + objDataFileName + "中!");  
        }  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
}
```

这句注释掉

这句取消注释

SerializeMultiTimes x

"C:\Program Files\
[0] 张三有30岁
[1] 张三有30岁
[2] 张三有30岁
false
false

多个相同内容的对象被序列化，反序列化时，会得到不同的对象实例。

如果对象序列化之后，状态有改变.....

```
private static void writePersonMultiTimes() {  
    //将被两次序列化的Person对象  
    Person person = new Person( name: "张三", age: 30);  
    try (ObjectOutputStream oos = new ObjectOutputStream(  
        new FileOutputStream(objDataFileName))) {  
        //将person对象写入输出流  
        oos.writeObject(person);  
        System.out.println(" 对象: " + person + "已写入到文件"  
            + objDataFileName + "中! ");  
        //同一对象，字段值修改!  
        person.setName("李四");  
        oos.writeObject(person);  
        System.out.println(" 对象: " + person + "已写入到文件"  
            + objDataFileName + "中! ");  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
}
```

```
private static void readPerson() {  
    try (ObjectInputStream ois = new ObjectInputStream(  
        new FileInputStream(objDataFileName))) {  
        //从输入流中读取一个Java对象，并将其强制类型转换为Person类  
        var person1 = (Person) ois.readObject();  
        System.out.println("第一次反序列化: " + person1);  
        var person2 = (Person) ois.readObject();  
        System.out.println("第二次反序列化: " + person2);  
        //检查是否引用同一个Person对象  
        System.out.println(person1 == person2);  
    } catch (IOException | ClassNotFoundException e) {  
        throw new RuntimeException(e);  
    }  
}
```

SerializeMultiTimes2.java

大坑!



```
SerializeMultiTimes2 x
"C:\Program Files\Java\jdk-17\bin\java.exe"
对象: "张三有30岁"已写入到文件multiTimes2.data中!
对象: "李四有30岁"已写入到文件multiTimes2.data中!
第一次反序列化: 张三有30岁
第二次反序列化: 张三有30岁
true
Process finished with exit code 0
```

从输出结果来看，程序中代码对Person对象name字段值的修改（“张三”改为“李四”），没有被保存!

同一个对象被序列化后，如果它的状态有改变，再次序列化时，由于Java不会重复序列化，它只是向流中输出了一个标识，这样一来，对状态的修改，将不会保存到流中。

那怎样基于Java的对象序列化特性，实现对“特定对象”状态的持续跟踪呢？



东边不亮西边亮!



不再尝试在同一个文件中保存不同时刻的对象状态，而是把这些状态有变化的对象，序列化到不同的文件中!

重构后的代码



将对象序列化和反序列化的代码，抽取为独立的方法，文件名作为方法参数.....

//将Person对象的当前值，序列化到指定的文件中

2 usages

```
static void writePersonToFile(Person person, String fileName) {  
    try (ObjectOutputStream oos = new ObjectOutputStream(  
        new FileOutputStream(fileName))) {  
        //将person对象写入输出流  
        oos.writeObject(person);  
        System.out.println(" 对象: " + person + " 已写入到文件"  
            + fileName + "中! ");  
    } catch (IOException e) {  
        System.out.println(e.getMessage());  
    }  
}
```

//从指定文件中反序列化Person对象

2 usages

```
private static Person readFromFile(String fileName) {  
    Person person = null;  
    try (ObjectInputStream ois = new ObjectInputStream(  
        new FileInputStream(fileName))) {  
        //从输入流中读取一个Java对象，并将其强制类型转换为Person类  
        person = (Person) ois.readObject();  
    } catch (IOException | ClassNotFoundException e) {  
        System.out.println(e.getMessage());  
    }  
    return person;  
}
```

SerializeMultiTimes3.java

测试重构是否达到预期效果.....



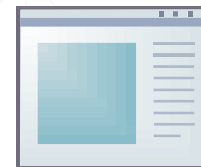
```
public static void main(String[] args) {  
    writePersonMultiTimes();  
    readPerson();  
}
```

```
private static void writePersonMultiTimes() {  
    //将被两次数序列化的Person对象  
    Person person = new Person( name: "张三", age: 30);  
    writePersonToFile(person, objDataFileName1);  
  
    //同一对象, 字段值修改!  
    person.setName("李四");  
    writePersonToFile(person, objDataFileName2);  
}
```

```
private static void readPerson() {  
  
    Person person1 = readFromFile(objDataFileName1);  
    System.out.println("第一次反序列化: " + person1);  
    Person person2 = readFromFile(objDataFileName2);  
    System.out.println("第二次反序列化: " + person2);  
    //检查两个变量是否引用同一个Person对象  
    System.out.println(person1 == person2);  
}
```

使用两个文件来保存
变化后的对象状态

运行截图



```
SerializeMultiTimes3 x
"C:\Program Files\Java\jdk-17\bin\java.exe"
对象: "张三有30岁"已写入到文件multiTimes1.dat中!
对象: "李四有30岁"已写入到文件multiTimes2.dat中!
第一次反序列化: 张三有30岁
第二次反序列化: 李四有30岁
false
Process finished with exit code 0
```

从输出结果来看，同一对象状态的变化过程确实被记录了下来，但反序列化时，你得到的是两个独立的Person对象，而并非最初的单个Person实例。

项目实战



在本讲示例中，展示了多对象序列化与单对象多次序列化的编程技巧，使用这些技巧，你现在应该可以开发一个支持“备份”与“回滚”的示例程序。



这个示例程序运行时，可以按照特定的时间间隔，采用序列化技术备份特定对象的状态，生成一系列特定文件名格式的文件。然后，用户只要指定一个备份文件名，就能重建当时的对象状态。



请动手一试！